ICML 2022 | From a Flat Datastore to an Automaton: Making Retrieval LMs Fast

RetoMaton skips up to 83% of nearest-neighbor searches with no loss in perplexity (CMU and AWS)

Retrieval-augmented language models earn their accuracy with a simple trick: at every step they look up the most similar past contexts in a large external datastore and blend those neighbors' next tokens into the prediction. kNN-LM showed this can sharply lower perplexity and adapt a model to a new domain without any retraining. The price is speed.

That nearest-neighbor search can fire at every single token, and it is far slower than the model's own forward pass. In practice it is the search, not the neural network, that dominates the cost, and it is what keeps these otherwise strong models out of production.

This ICML 2022 paper from Carnegie Mellon and AWS, RetoMaton (short for "retrieval automaton"), attacks that cost directly. Its observation is that a flat datastore throws away structure the retrieval could reuse, and its fix is to compile the datastore into a weighted finite automaton that the model walks alongside decoding. The payoff: up to 83% of nearest-neighbor searches skipped with no loss in perplexity, or up to 1.85 lower perplexity when the search budget is kept.

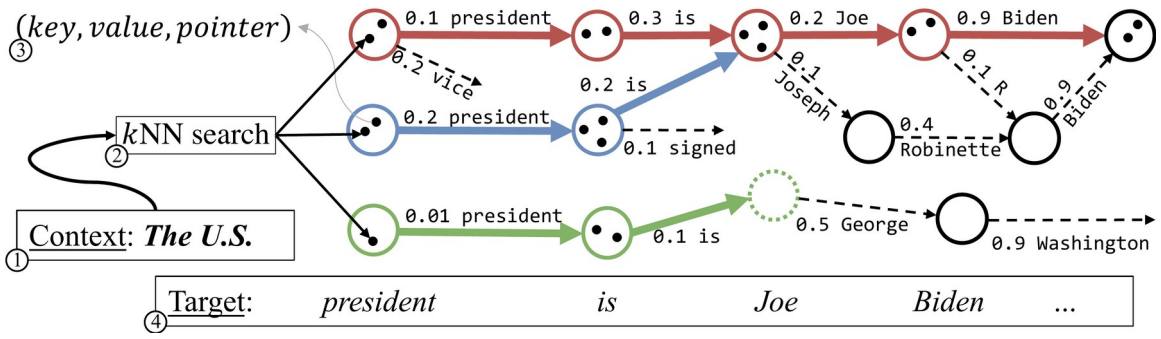

Figure 1. RetoMaton at inference. From the context ("The U.S."), a single kNN search seeds several automaton states (the colored circles), each a cluster of datastore entries that share outgoing pointers. The model then follows those pointers in parallel as it decodes "president is Joe Biden," reusing retrieval across steps instead of searching from scratch at every token. Dashed arrows are transitions that were allowed but not taken.

Paper: https://arxiv.org/abs/2201.12431 · Code: https://github.com/neulab/retomaton

The bottleneck: searching a flat datastore

kNN-LM stores one entry per training token: a key (the LM's hidden state at that position) and a value (the token that actually followed). At test time it embeds the current context, finds the k nearest keys, and turns their values into a second next-token distribution that is interpolated with the base LM. Because there is one entry per token, and hundreds of millions of tokens, each search is expensive, and the model runs one for every token it generates.

The authors' key observation is that this flat list ignores two kinds of structure. First, entries are correlated in time: if a retrieved entry was useful now, the entry that came right after it in the original corpus is very likely useful next. Second, entries with nearby keys tend to be followed by the same token. A flat datastore rediscovers both facts from scratch on every step. Prior work such as Adaptive Retrieval (AdaptRet) learns when to skip a search, but when it skips it falls back on the base LM alone and discards the retrieval signal, which hurts most in exactly the domains where retrieval helps.

Two changes turn the datastore into an automaton

RetoMaton makes two unsupervised changes to the datastore, both illustrated in Figure 2. First, it saves a pointer from every entry to its successor, the entry that came next in the text. Second, it clusters entries with similar key vectors into states, and every state inherits the outgoing pointers of all its members. Together, the pointers and the clusters turn the flat list into a weighted finite automaton whose states are groups of similar contexts and whose transitions are the "what came next" links.

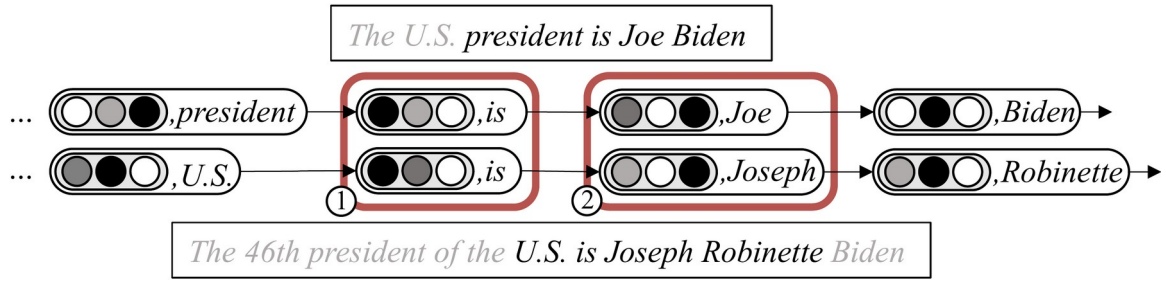

Figure 2. Building the automaton, unsupervised. Two sentences share the continuation "... is ...". Each entry is a (key, value) pair; a pointer links every entry to its successor in the text, and entries with close keys are merged into shared states (1 and 2). Merging lets one search cover many contexts at once, and the pointers let the model step forward without searching again.

At inference the model keeps a small set of active states and traverses the automaton in parallel with the LM (Figure 1). To move forward it simply follows the pointers of entries whose value matches the token just generated, which is essentially free. The transition weights are not fixed: they are computed on the fly from the distance between the current hidden state and the entries in each state, then interpolated with the base LM as p(w | c, S) = lambda * p_auto(w | c, S) + (1 - lambda) * p_LM(w | c), so retrieval still adapts to the live context. A full kNN search is triggered only when the number of valid onward transitions falls below a threshold tau. That threshold is the single knob that trades accuracy (restart often, search more) against speed (search rarely). Building the automaton needs no training data and works whether it is built from the model's own training corpus or from a brand-new domain.

Results: fewer searches, same or better perplexity

On WikiText-103 (in-domain, 103M training tokens, a 247M-parameter Transformer base LM, datastore clustered into 1M states), RetoMaton matches kNN-LM's perplexity while skipping 81% of the searches. More striking, even at FoSS=0, when it searches on every step just as kNN-LM does, it still lowers perplexity, from 16.65 (kNN-LM) and 16.35 (AdaptRet) down to

16.08 (Table 1). FoSS, the Fraction of Saved Searches, serves as a hardware-independent proxy for wall-clock savings.

Table 1. WikiText-103 in-domain perplexity at FoSS=0 (no searches saved). Lower is better.

Model	Perplexity
kNN-LM	16.65
AdaptRet	16.35
RetoMaton (pointers only)	16.12
RetoMaton	16.08

The gains are larger and more robust under domain shift. Building the datastore from Law-MT (19M tokens, a 656M-parameter base LM, 200K states), RetoMaton drops perplexity from 12.34 (kNN-LM) to 10.49 at FoSS=0, and as more searches are saved its perplexity climbs only gently while plain kNN-LM's rises sharply, because kNN-LM has nothing to fall back on but the base model. With a base LM already fine-tuned on Law-MT, RetoMaton still adds the largest gain (Table 2), cutting perplexity from 8.61 to 7.10, a 17.5% relative reduction, against 7.93 for kNN-LM and 7.81 for AdaptRet.

Table 2. Law-MT with a base LM fine-tuned on Law-MT. Perplexity at two saving levels; lower is better.

Model	FoSS=0	FoSS=0.5
fine-tuned LM	8.61	-
kNN-LM	7.93 (-7.9%)	8.25 (-4.2%)
AdaptRet	7.81 (-9.2%)	7.91 (-8.1%)
RetoMaton	7.10 (-17.5%)	7.15 (-17.0%)

Parentheses show the relative reduction over the fine-tuned LM. RetoMaton keeps almost all of its gain even after saving half of the searches.

What does the work: pointers or clusters?

An ablation separates the two ingredients. Pointers alone, with no clustering, already beat every baseline and match kNN-LM while saving more than 60% of searches, so the "what came next" links carry most of the benefit at low saving rates: the pointers-only variant reaches 16.12 at FoSS=0, essentially tied with the full model's 16.08. Clustering helps mainly at high FoSS (roughly 0.7 and above), where merged states allow longer search-free stretches. On cluster count, 500K and 1M states behave similarly while 100K is too coarse, and a cheaper greedy merge wins at FoSS=0 but fades as more searches are saved.

Why it matters

The lasting idea is that a retrieval datastore has structure worth compiling. By linking consecutive entries with pointers and grouping similar ones into states, RetoMaton lets a language model carry retrieval forward in time instead of searching from scratch at every token. The construction is unsupervised, model-agnostic, transfers across domains, and unifies token-, chunk-, and sequence-level retrieval in one mechanism.

The honest boundary: RetoMaton reduces how often the model searches, not the cost of a single search or the memory of storing hundreds of millions of entries, so the datastore still has to live somewhere and be compiled once before it pays off. Within those limits, it is a clean, training-free way to make retrieval-based language models substantially cheaper to run.